Yaml files specific to “kind”

In Kubernetes, "kinds" refer to the different types of resources or objects that can be managed within a cluster. These resources define the various elements that Kubernetes handles in order to deploy, manage, and scale applications. Below are some of the main Kubernetes kinds:

1. Pod

- **Definition:** The basic unit of deployment in Kubernetes. A pod is a group of one or more containers that are deployed together on the same host.
- **Use:** Pods are used to run containers in a Kubernetes environment. Typically, a pod will contain one container, but you can run multiple containers if they need to share storage, networking, or other resources.

```
apiVersion: v1
kind: Pod
metadata:
name: pathnex-pod
spec:
containers:
- name: nginx
image: nginx:latest
ports:
- containerPort: 80
```

PATHNEX
Careers, Building Futures
Shaping

2. Deployment

- **Definition:** A higher-level abstraction that manages a set of pods. A Deployment ensures that a specified number of pod replicas are running at all times.
- **Use:** Deployments are used to manage stateless applications and to provide rolling updates or rollbacks.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pathnex-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: pathnex
  template:
    metadata:
      labels:
        app: pathnex
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

3. ReplicaSet

- **Definition:** Ensures that a specified number of pod replicas are running at any given time.
- **Use:** Although ReplicaSets are commonly managed by Deployments, they can also be used directly to manage the scaling of a set of pods.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: pathnex-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: pathnex
  template:
    metadata:
      labels:
        app: pathnex
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

PATHNEX
Shaping Careers, Building Futures

4. StatefulSet

- **Definition:** A type of resource used to manage stateful applications. Unlike Deployments, which are stateless, StatefulSets maintain the identity and storage of pods across restarts.
- **Use:** StatefulSets are used for applications that require persistent storage and stable network identities, such as databases.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: pathnex-statefulset
spec:
  serviceName: "pathnex"
  replicas: 3
  selector:
    matchLabels:
      app: pathnex
  template:
    metadata:
      labels:
        app: pathnex
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

5. DaemonSet

- **Definition:** Ensures that a specific pod is running on all or some nodes in the cluster.
- **Use:** DaemonSets are useful for running background tasks or services, like monitoring agents, logging agents, or network proxies, on all nodes.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: pathnex-daemonset
spec:
  selector:
    matchLabels:
      app: pathnex
  template:
    metadata:
      labels:
        app: pathnex
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

6. Job

- **Definition:** A resource for running batch or one-time tasks that are expected to complete. A Job will ensure that a specified number of pods successfully terminate.
- **Use:** Jobs are ideal for tasks that need to run to completion, like data processing, backups, or batch jobs.

PATHNEX
Shaping Careers, Building Futures

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pathnex-job
spec:
  template:
    spec:
      containers:
      - name: busybox
        image: busybox
        command: ["sh", "-c", "echo Hello, Kubernetes!"]
      restartPolicy: Never
```

7. CronJob

- **Definition:** A scheduled Job that runs periodically at specified times or intervals (similar to cron jobs in Unix/Linux).
- **Use:** Useful for tasks that need to be executed on a recurring basis, such as backups or report generation.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: pathnex-cronjob
spec:
  schedule: "*/5 * * * *"  # Runs every 5 minutes
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: busybox
            image: busybox
            command: ["sh", "-c", "echo Hello, Kubernetes!"]
          restartPolicy: OnFailure
```

8. Service

- **Definition:** A resource that defines a logical set of pods and a policy by which to access them. Services expose your pods to internal or external traffic.
- **Use:** Services provide a stable IP address and DNS name for accessing a group of pods. They can expose services externally (via `LoadBalancer` or `NodePort`) or internally within the cluster (`ClusterIP`).

```
apiVersion: v1
kind: Service
metadata:
  name: pathnex-service
spec:
  selector:
    app: pathnex
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

PATHNEX
Shaping Careers, Building Futures

9. Ingress

- **Definition:** A resource that manages external access to services in a cluster, typically HTTP/S.
- **Use:** Ingress allows you to define rules for routing external HTTP/S traffic to specific services within the cluster. It can also manage SSL termination and URL path-based routing.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: pathnex-ingress
spec:
  rules:
  - host: pathnex.com
```

```
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: pathnex-service
            port:
              number: 80
```

10. ConfigMap

- **Definition:** A way to store non-confidential data in key-value pairs that can be accessed by pods.
- **Use:** ConfigMaps are used to store configuration files or environment variables, which can be mounted into pods as files or environment variables.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: pathnex-configmap
data:
  app-config: |
    key1: value1
    key2: value2
```

PATHNEX
Shaping Careers, Building Futures

11. Secret

- **Definition:** Similar to ConfigMap but designed to store sensitive information like passwords, OAuth tokens, or SSH keys.
- **Use:** Secrets are used to safely store sensitive data and can be injected into pods as environment variables or files.

```
apiVersion: v1
kind: Secret
metadata:
  name: pathnex-secret
type: Opaque
data:
  username: YWRtaW4=  # base64 encoded "admin"
  password: cGFzc3dvcmQ=  # base64 encoded "password"
```

12. PersistentVolume (PV)

- **Definition:** A piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using a StorageClass.
- **Use:** PVs are used to manage storage resources independently of the lifecycle of a pod.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pathnex-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/tmp/data"
```

13. PersistentVolumeClaim (PVC)

- **Definition:** A request for storage by a user. A PVC allows a pod to access a PersistentVolume.
- **Use:** PVCs are used to claim storage from available PersistentVolumes.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pathnex-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

PATHNEX
Shaping Careers, Building Futures

14. Namespace

- **Definition:** A way to divide cluster resources between multiple users or teams.
- **Use:** Namespaces are used to organize and isolate resources within a cluster, providing a scope for resource names.

```
apiVersion: v1
kind: Namespace
metadata:
  name: pathnex-namespace
```

15. NetworkPolicy

- **Definition:** Defines how groups of pods are allowed to communicate with each other and other network endpoints.
- **Use:** NetworkPolicies are used to restrict traffic between pods based on labels and other criteria.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: pathnex-network-policy
spec:
  podSelector:
    matchLabels:
      app: pathnex
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: frontend
```

16. ResourceQuota

- **Definition:** Defines constraints on the amount of resources (CPU, memory, etc.) that can be used within a namespace.
- **Use:** ResourceQuotas are used to manage and limit resource usage across a namespace.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: pathnex-resource-quota
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
```

PATHNEX
Shaping Careers, Building Futures

17. HorizontalPodAutoscaler (HPA)

- **Definition:** Automatically scales the number of pods in a deployment, replica set, or stateful set based on CPU utilization or other custom metrics.
- **Use:** HPA is used to automatically adjust the number of replicas to match workload demands.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: pathnex-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: pathnex-deployment
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
```

```
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

18. CustomResourceDefinition (CRD)

- **Definition:** Allows you to define your own custom resources in Kubernetes. A CRD extends the Kubernetes API to include new kinds of resources.
- **Use:** CRDs enable users to manage resources that are specific to their applications, like defining a `Database` resource or a `Backup` resource.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.pathnex.com
spec:
  group: pathnex.com
  names:
    kind: Database
    plural: databases
    singular: database
    shortNames:
      - db
  scope: Namespaced
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              engine:
                type: string
                enum:
                  - mysql
                  - postgresql
```

PATHNEX
Shaping Careers, Building Futures

19. PodDisruptionBudget (PDB)

- **Definition:** A policy that defines the minimum number of pods that must be available during voluntary disruptions (like node maintenance).
- **Use:** PDBs are used to maintain the availability of applications during updates or maintenance.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pathnex-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: pathnex
```

20. Volume

- **Definition:** An abstraction over storage resources, such as PersistentVolumes or ephemeral storage, that can be used by containers within a pod.
- **Use:** Volumes are used to provide a way to persist data or share data between containers in a pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-volume
spec:
  containers:
  - name: nginx
    image: nginx:latest
    volumeMounts:
    - name: pathnex-volume
      mountPath: /usr/share/nginx/html
  volumes:
  - name: pathnex-volume
    emptyDir: {}
```

PATHNEX
Shaping Careers, Building Futures

These are the most common Kubernetes kinds, but Kubernetes is extensible, and you can also define your own custom kinds using CustomResourceDefinitions (CRDs) to support specific use cases. You can customize them based on your application needs, including adding environment variables, setting resource limits, or configuring specific behaviors.